

CO5 – ASSESSMENT TOOL 2 (AT2)

Student Name : Shaik Ahamed Ali
Registration Number : 192425106
Course Code & Name : MLA0302 - Reinforcement Learning

Reinforcement Learning — Scenario-Based Assessment (Policy-Based RL Methods)

Q1. Personalized Mobile Notification Engine using Reinforcement Learning

Understanding of the Scenario

A mobile app wants to boost user engagement by delivering notifications at the moment a user is most likely to respond, instead of at fixed or random times. This is a sequential decision-making problem — every notification decision influences future user trust and app usage — which makes it a natural fit for Reinforcement Learning (RL), where an agent learns an optimal notification policy purely through trial, feedback and reward.

Key Issues Identified

The core issues are: (a) identifying which contextual signals genuinely predict engagement, (b) avoiding notification fatigue that causes users to disable alerts or uninstall the app, (c) balancing short-term clicks against long-term retention, and (d) handling a highly non-stationary environment since user habits shift over time (weekday vs weekend, festivals, life events).

Application of RL Concepts

- **State (S):** Time of day/day of week, recent in-app activity, last notification's outcome (opened/ignored/dismissed), user's historical response rate, current location/context, device idle status.
- **Action (A):** Send / don't send a notification now; choose the content category (offer, reminder, social update); choose notification channel (push, in-app banner); choose the exact send time within a window.
- **Reward (R):** +1 (or scaled positive) if the user opens/clicks the notification within a short window; small negative reward for a notification that is dismissed unread; a larger negative penalty if the user disables notifications or the app is uninstalled soon after.

Decision / Solution & Justification

A contextual-bandit or Deep Q-Network policy can be trained offline on historical logs and refined online. The agent should be constrained with a maximum notifications-per-day cap and reward shaping that discounts click reward if it is followed by disengagement, so the policy learns patience rather than notification spamming.

Diagram

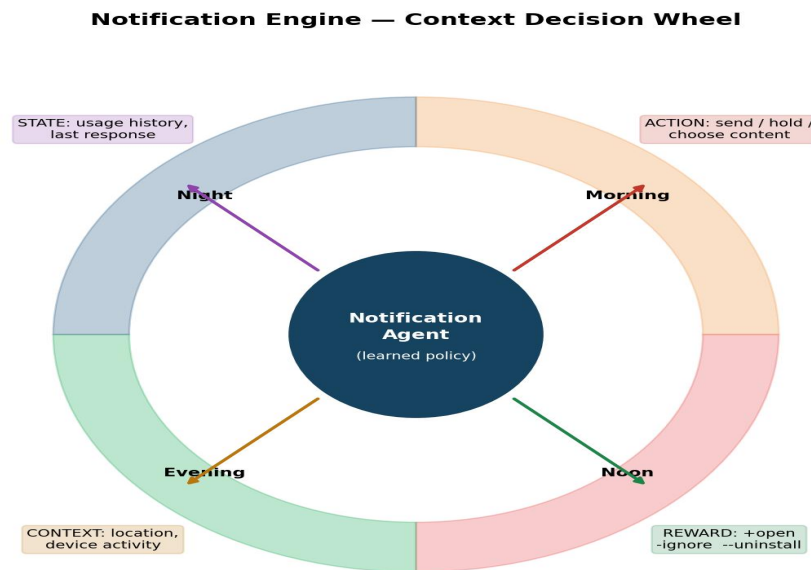


Fig 1.1 — Radial decision-wheel: time-of-day states driving the notification agent's action/reward cycle

Q2. Exploration vs Exploitation Trade-off in Warehouse Robot Navigation

Understanding of the Scenario

A warehouse robot must find the shortest, safest delivery path while avoiding obstacles. The robot's policy is trained through RL, where it explores different paths to discover shorter routes and exploits the best-known route once learned. When exploration is insufficient during training, the robot converges early to a sub-optimal — but 'safe-looking' — path and keeps repeating it, resulting in longer real-world delivery times.

Key Issues Identified

Key issues: premature convergence to a locally optimal path, poor generalisation to new obstacle layouts not seen during limited exploration, risk of collisions if exploration is too aggressive, and slow re-adaptation when the warehouse layout changes (new shelves, temporary blockages).

Application of RL Concepts

- Why it affects performance: Too little exploration → robot never discovers shorter alternate corridors, so it exploits a mediocre path repeatedly. Too much exploration → robot wastes time/battery testing random moves and may increase collision risk.
- Effect of imbalance: Low exploration = longer average delivery path length and stagnant learning curve; excessive exploration = unstable, inefficient early performance and unsafe navigation near obstacles.

Decision / Solution & Justification

Improvements: (1) use an epsilon-greedy strategy with epsilon decaying over training episodes so exploration is high early and low later; (2) apply Upper-Confidence-Bound (UCB) or softmax action selection to explore promising-but-uncertain paths more intelligently than pure randomness; (3) use curiosity-driven / intrinsic-reward exploration to encourage visiting unexplored warehouse zones; (4) periodically re-inject exploration (epsilon-restart) whenever the warehouse map changes, so the robot re-discovers better routes instead of freezing on an outdated policy.

Diagram

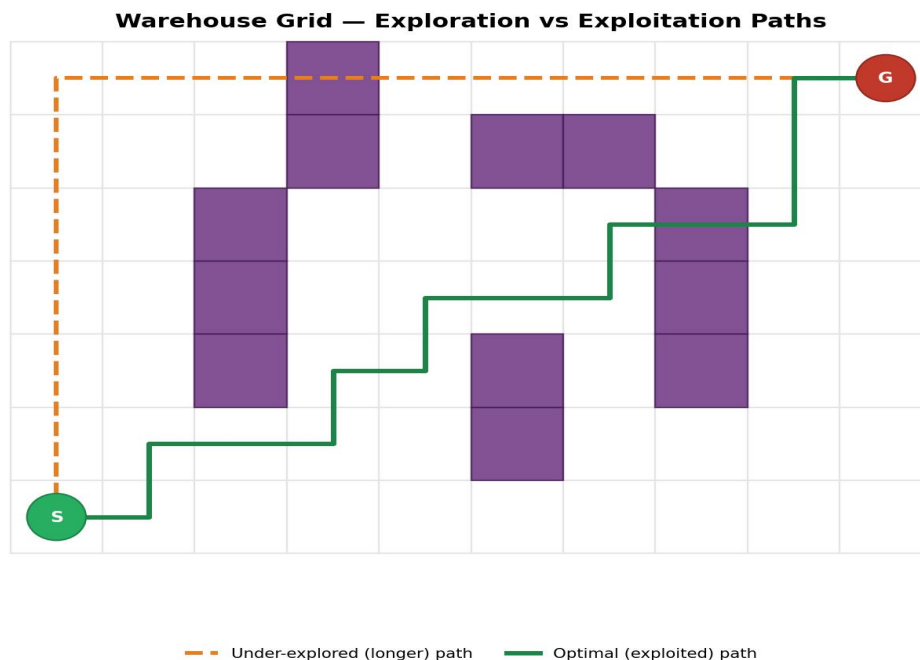


Fig 2.1 — Warehouse grid map comparing an under-explored (longer) path with the optimal exploited path

Q3. RL for Hospital Appointment Scheduling

Understanding of the Scenario

A hospital wants to reduce patient waiting time and improve utilisation of doctors and rooms by optimising how appointments are scheduled. RL suits this problem because scheduling is sequential and dynamic — each new booking decision affects the state of the queue and resource availability for every future decision, which is exactly the kind of long-horizon, feedback-driven problem RL is designed to solve.

Key Issues Identified

Two major challenges stand out. Delayed rewards: the true benefit of a scheduling decision (e.g., avoiding an overload three hours later) is only observed much later, making credit assignment difficult. Ethical considerations: the reward function must not implicitly learn to deprioritise certain patient groups (age, insurance type, condition severity) purely because doing so reduces average wait time — this could create unfair or even dangerous prioritisation.

Application of RL Concepts

- State (S): Doctor/room availability, current queue length, patient urgency level, historical no-show probability, time since last appointment.
- Action (A): Assign a specific time slot, place on waitlist, escalate/prioritise an urgent case, reassign to another available doctor.
- Reward (R): + for reduced average waiting time and high resource utilisation, – for overbooking, missed urgent cases, or unfair prioritisation patterns.

Decision / Solution & Justification

RL is well suited here provided reward shaping includes fairness constraints (e.g., a hard priority rule for emergencies that RL cannot override) and reward smoothing/eligibility traces to handle delayed outcomes. A human-in-the-loop review of the learned policy before deployment is recommended to catch ethically biased behaviour.

Diagram

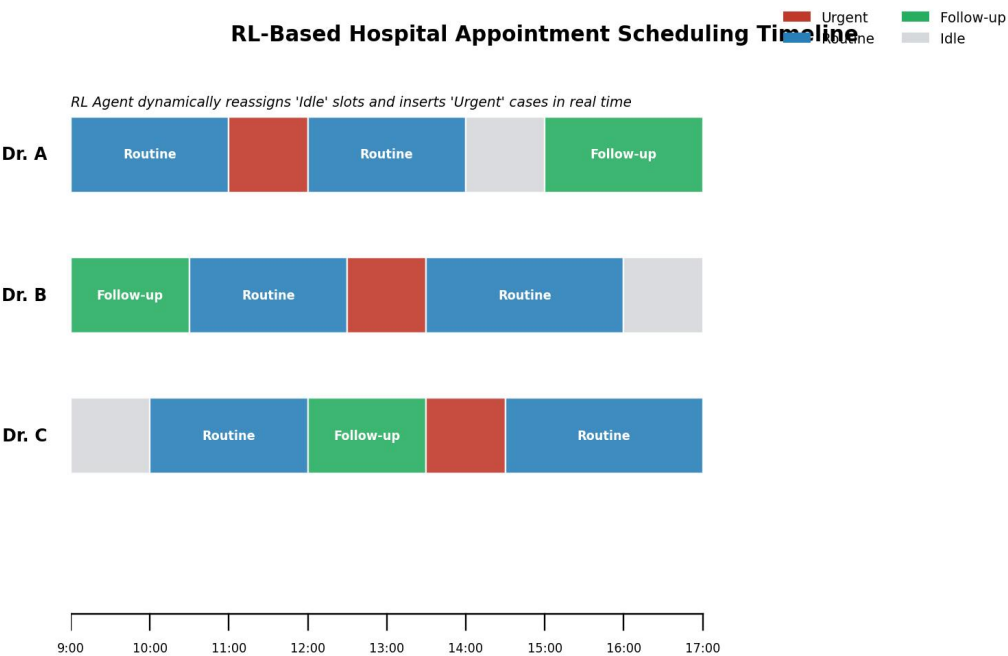


Fig 3.1 — Gantt-style doctor scheduling timeline showing urgent, routine, follow-up and idle slots

Q4. RL Model for Smart Home Energy Management

Understanding of the Scenario

A smart home system should automatically control lights, fans and air conditioners to cut electricity usage while keeping residents comfortable, based on presence and weather. This is a continuous control problem with competing objectives (comfort vs. savings), which is well handled by RL because the agent can learn trade-offs directly from reward feedback instead of relying on fixed rules.

Key Issues Identified

Model design must balance conflicting goals — minimising energy vs. maintaining comfort — while adapting to seasonal weather changes and irregular occupancy patterns, without causing large, sudden discomfort to residents.

Application of RL Concepts

- State Space: Room occupancy (sensor), indoor temperature/humidity, outdoor weather forecast, time of day, current appliance status, historical usage pattern.
- Action Space: Turn light/fan/AC ON or OFF, adjust thermostat set-point, dim lighting level, switch to eco-mode.

- Reward Structure: + reward proportional to energy saved while occupant comfort stays within an acceptable range; – penalty for excess consumption when room is empty, and – penalty for discomfort (temperature/light outside comfort band).
- Suitable Algorithm: Deep Q-Network (DQN) for discrete on/off decisions, or Proximal Policy Optimization (PPO)/Deep Deterministic Policy Gradient (DDPG) if thermostat control is treated as continuous — PPO/DDPG handle the continuous action space of temperature adjustment better than tabular Q-learning.

Decision / Solution & Justification

A PPO-based agent with a weighted reward ($\text{comfort_weight} \times \text{comfort_score} + \text{energy_weight} \times \text{savings_score}$) allows the household to tune the comfort–savings trade-off, and the policy can be retrained seasonally as weather patterns change.

Diagram

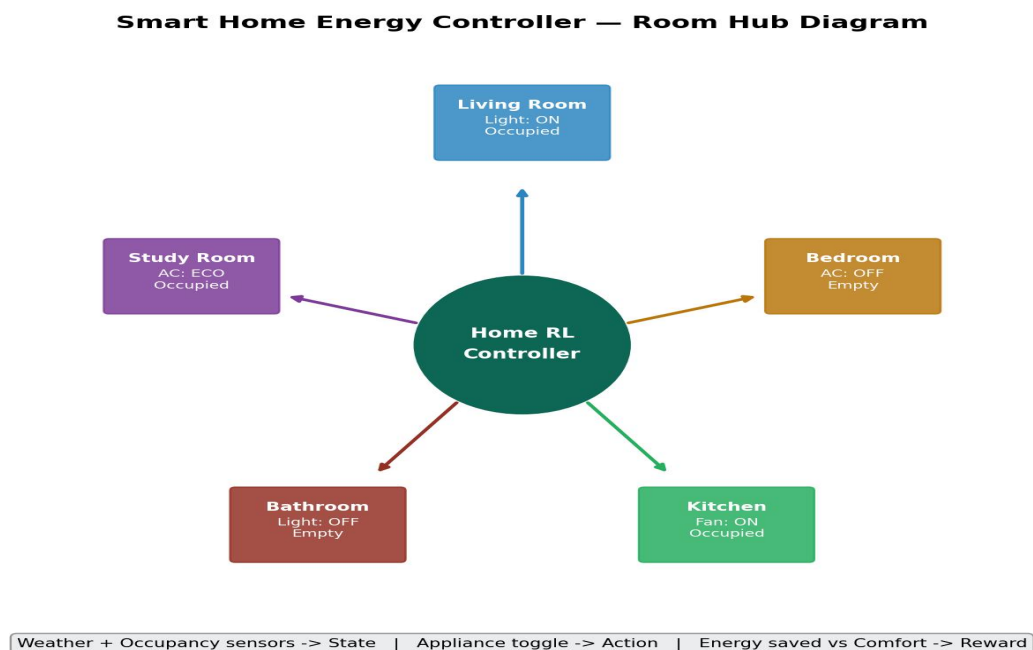


Fig 4.1 — Hub-and-spoke floor-plan diagram: home controller managing each room's appliance state

Q5. Reward Design in Online Gaming Matchmaking

Understanding of the Scenario

An online gaming platform uses RL to pair players of similar skill so that matches stay competitive and engaging. Since matchmaking quality directly depends on how the reward function is defined, reward design is the single most important lever for the system's behaviour.

Key Issues Identified

If the reward purely maximises 'match speed' (queue time), the agent learns to pair players quickly regardless of skill gap, producing lopsided, unfair matches. If the reward purely maximises 'skill closeness', queue times may balloon at odd hours when few similar-skill players are online, frustrating players who then leave the platform. An overly narrow reward (e.g., based only on win/loss closeness) can also ignore other engagement signals such as match duration or post-match ratings, leading to biased matching that repeatedly favours certain player segments.

Application of RL Concepts

- **Balanced Reward Design:** Combine multiple weighted terms: + for small skill-rating gap between teams, + for reasonable queue wait, + for post-match satisfaction/rating, and – for one-sided score outcomes.
- **Biased Reward Example:** Reward based only on 'time-to-match' → agent exploits this by ignoring skill difference entirely, causing lopsided games and player attrition.

Decision / Solution & Justification

The recommended design is a multi-objective reward (skill-fit + wait-time penalty + post-game feedback) rather than a single metric, along with periodic audits of match outcomes across player segments to detect and correct systematic bias in the learned matchmaking policy.

Diagram

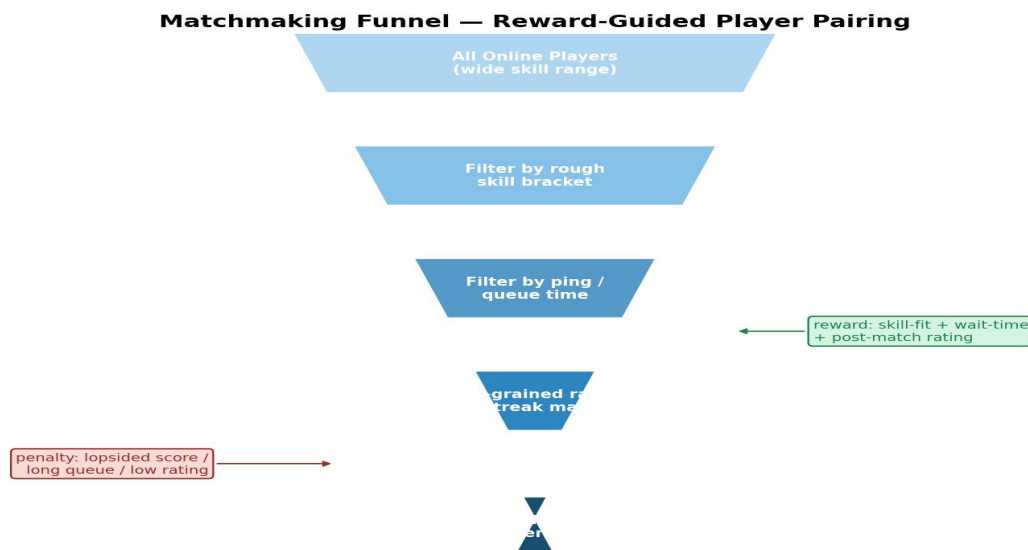


Fig 5.1 — Matchmaking funnel: player pool narrowed by skill, ping and rating filters to a final pair

Q6. RL for Financial Trading System

Understanding of the Scenario

A trading system uses RL to decide buy/sell/hold actions from market signals, treating trading as a sequential decision problem under uncertainty — the agent must act now while the true outcome (profit/loss) is only realised later, sometimes long after the decision was made.

Key Issues Identified

The market is highly volatile and non-stationary, so a policy trained on past data may not generalise to future regimes ('distribution shift'). Delayed and noisy rewards make credit assignment hard — a profitable trade might be the result of luck rather than a genuinely good policy, and vice-versa.

Application of RL Concepts

- State (S): Historical price/volume data, technical indicators (moving averages, RSI), current portfolio position, volatility index, macro/news sentiment.
- Action (A): Buy, sell, hold, or adjust position size (partial buy/sell).
- Reward (R): + for realised profit (often risk-adjusted, e.g., Sharpe-ratio based), – for realised loss, with an additional penalty for excessive drawdown or over-leveraged exposure.

Decision / Solution & Justification

Risk evaluation: in volatile markets, delayed rewards can cause the agent to take excessively risky short-term positions chasing sparse long-term profit signals. This is mitigated by risk-adjusted reward shaping, position-size limits, and using techniques like Conservative Q-Learning or ensemble policies to reduce over-fitting to a single historical regime.

Diagram

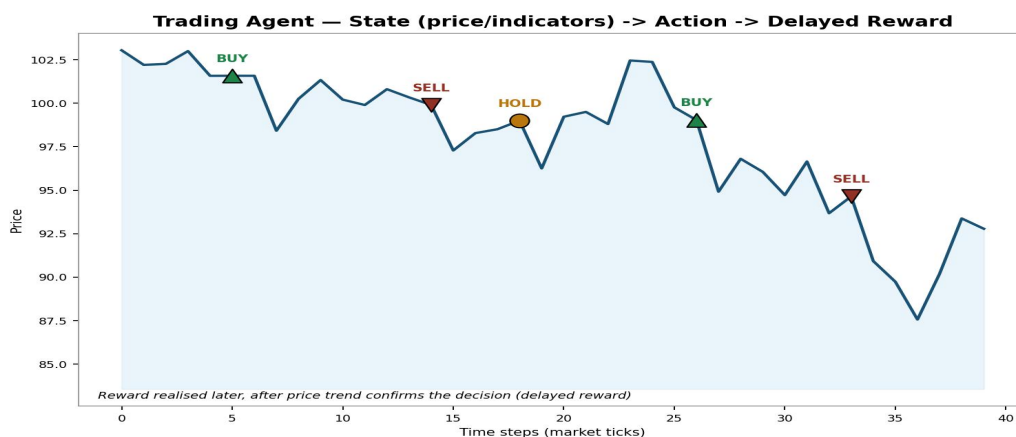


Fig 6.1 — Price-time chart marking Buy/Sell/Hold actions and the delayed-reward effect

Q7. RL Framework for Drone Delivery Route Optimization

Understanding of the Scenario

A drone delivery company needs routes that are both efficient and legally/safely compliant, avoiding restricted airspace. This requires an RL framework where safety constraints are enforced as strongly as efficiency goals.

Key Issues Identified

Balancing delivery speed against hard safety constraints (no-fly zones, weather, battery limits) is the central challenge; a purely efficiency-driven reward could tempt the agent to cut through restricted airspace if it isn't penalised heavily enough.

Application of RL Concepts

- State (S): GPS position, altitude, battery level, wind conditions, distance to destination, live map of restricted/no-fly zones.
- Action (A): Move forward, turn, ascend, descend, hover, or re-route around an obstacle/zone.
- Reward (R): + for successful, on-time, safe delivery; small – per unit distance/time (efficiency cost); large – (heavy penalty) for entering restricted airspace or violating altitude limits.

Decision / Solution & Justification

A constrained-RL or safe-RL formulation is justified: restricted-airspace violation is modelled as a hard constraint with a very large negative reward (or an outright action mask that disallows entering the zone), ensuring the learned policy treats safety as non-negotiable rather than something to be traded off against speed. This guarantees the drone converges to policies that are efficient only within the safe operating region.

Diagram

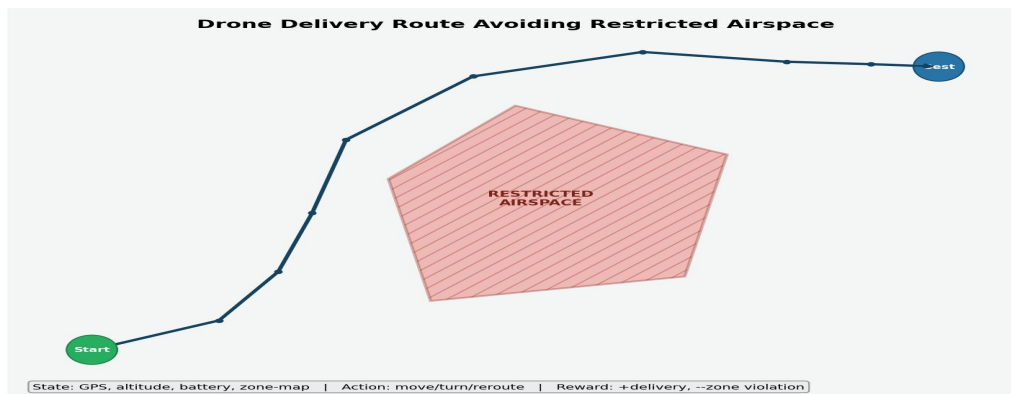


Fig 7.1 — Route map: drone path re-routing around a restricted-airspace zone to reach its destination

Q8. RL for Personalized Online Learning Paths

Understanding of the Scenario

A university uses RL to adapt each student's learning path — sequencing topics, difficulty, and content type — to maximise learning outcomes. Because student engagement evolves over a course, RL's sequential adaptation is a strong fit compared to static, one-size-fits-all curricula.

Key Issues Identified

Reward design directly shapes what the system optimises for: if reward is based only on quiz scores, the agent may push students toward easy content to farm high scores rather than genuine mastery. If based only on time-on-platform, it may addictively over-engage students rather than teach effectively.

Application of RL Concepts

- **Balanced Reward Design:** Combine mastery improvement (pre/post assessment gain), appropriate challenge level (not too easy/hard), and sustained engagement (completion rate) rather than a single proxy metric.
- **Ethical Considerations:** The system must avoid manipulative engagement tactics (reward hacking on 'time spent' rather than learning), must not create unfair learning-path disparities among student groups, should keep some transparency/human oversight over content choices, and must protect student data privacy used to build the state representation.

Decision / Solution & Justification

A defensible design uses mastery-gain as the primary reward signal, with engagement as a secondary/regularising term, plus periodic instructor review of the recommended learning paths to ensure the policy is pedagogically sound and ethically fair across student demographics.

Diagram

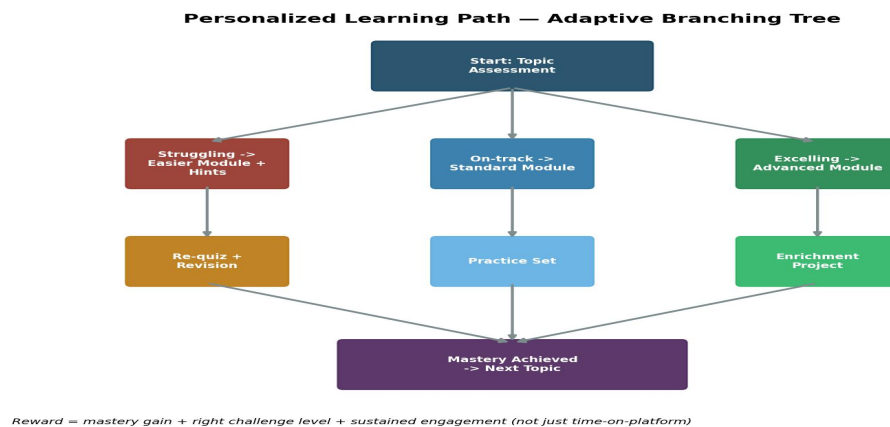


Fig 8.1 — Branching decision tree: adaptive learning path based on assessment performance

Q9. RL for Smart Traffic Light Control

Understanding of the Scenario

A traffic authority uses RL to dynamically control signal timing at busy intersections to reduce congestion, instead of relying on fixed-cycle timers. The agent observes traffic conditions and adapts green/red durations in real time.

Key Issues Identified

Beyond simple congestion minimisation, safety-critical constraints must be respected: emergency vehicles need guaranteed right-of-way, and pedestrians must have safe, adequately-timed crossing windows — these cannot be sacrificed even if doing so would reduce average vehicle wait time.

Application of RL Concepts

- **Constraint Handling:** Emergency-vehicle presence triggers an override/interrupt in the state that forces an immediate green phase on that vehicle's approach (implemented as a hard rule or an extremely large reward/priority term, not left purely to learned trade-off). Pedestrian-safety is enforced via a minimum guaranteed crossing time built into the action space, so the agent can only choose among signal timings that satisfy this minimum.
- **Reward Structure:** + for reduced average vehicle wait/queue length, + for honouring emergency priority quickly, – heavy penalty for unsafe/incomplete pedestrian crossing windows or blocking an emergency vehicle.

Decision / Solution & Justification

With these constraints embedded, the RL policy interprets congestion reduction as a secondary objective achieved within the safety envelope — simulations of such constrained traffic-RL systems typically show meaningfully shorter average commute/queue times at busy intersections compared to static timers, while emergency response time and pedestrian safety are preserved.

Diagram

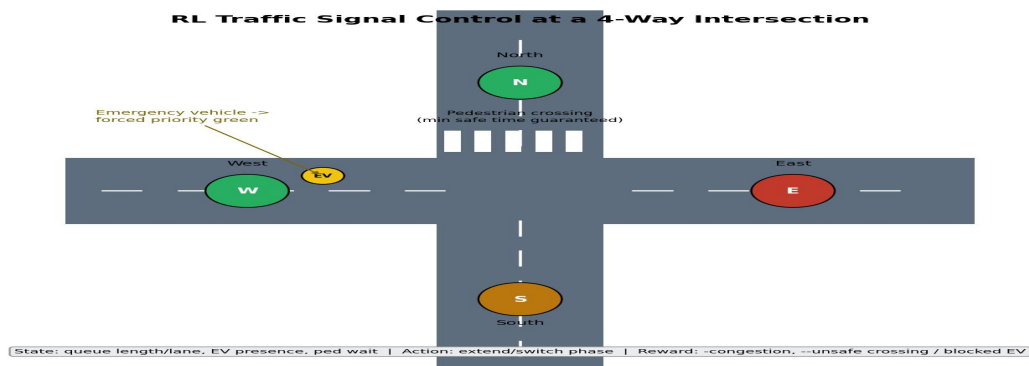


Fig 9.1 — 4-way intersection layout showing per-direction signals, emergency priority and pedestrian crossing

Q10. Traditional Optimization vs Constrained RL for Manufacturing Scheduling

Understanding of the Scenario

A manufacturing plant must schedule machines under limited resources (materials, tools, manpower) to maximise throughput without violating capacity constraints. Both classical optimisation and constrained RL are candidate approaches.

Key Issues Identified

Traditional methods (Linear Programming, Mixed-Integer Programming, heuristic dispatch rules) assume a largely static, well-characterised problem and re-solve from scratch when conditions change, which is computationally expensive and slow to react to real-time disruptions (machine breakdowns, urgent orders).

Application of RL Concepts

- Traditional Optimization: Provides mathematically optimal or near-optimal solutions for a fixed, well-defined problem instance; struggles with real-time adaptability and scales poorly as complexity/uncertainty grows; requires the problem to be re-modelled and re-solved whenever conditions shift.
- Constrained RL: Learns a policy that generalises across many production states and can react in real time to unexpected events (breakdowns, rush orders) once trained; constraints (resource limits) are encoded via penalty terms or safe-action masking so the agent never proposes an infeasible schedule.

Decision / Solution & Justification

For dynamic, uncertain production environments, constrained RL provides better adaptability because it learns a reusable policy that reacts instantly to changing plant conditions, whereas traditional optimisation gives more provably optimal results for static, well-understood scheduling instances but must be re-run for every change. In practice, a hybrid approach — traditional optimisation for baseline/offline planning and constrained RL for real-time re-scheduling — offers the best of both.

Diagram

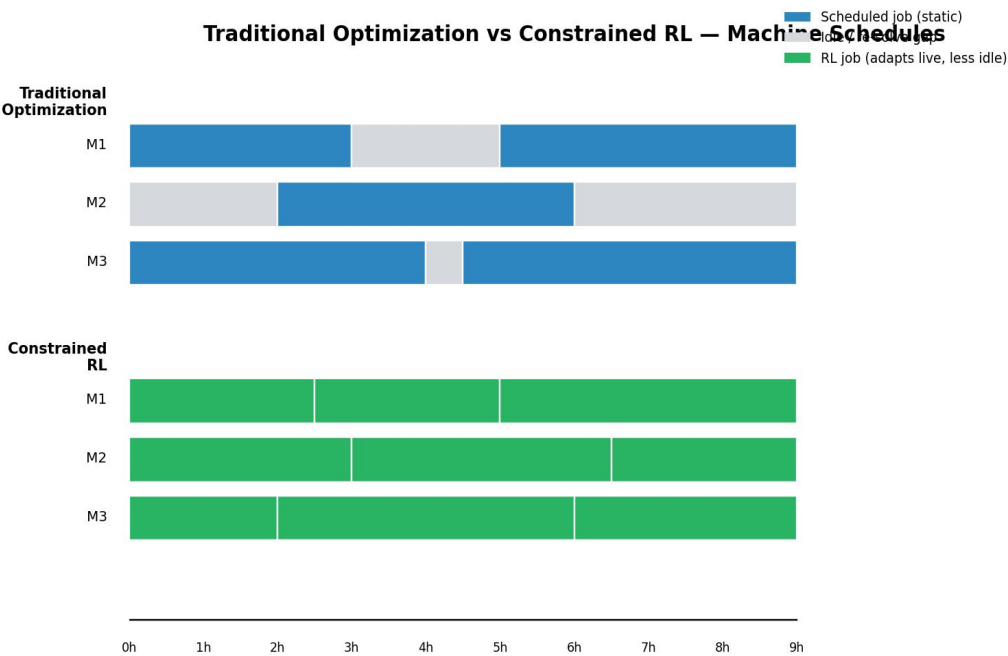


Fig 10.1 — Gantt comparison: static traditional-optimization schedule vs adaptive constrained-RL schedule